

Manual de Migración — Integrar Impresiones con PrinterServices

Guía paso a paso para migrar cualquier evento de impresión (`List<Impresion>`) al flujo de PrinterServices. Cubre modo **SERVIDOR** y modo **CLIENTE**. Ejemplo práctico: **Anulación de Pedidos** (`btnEliminar_Click` → `FragmentDetallePedidoPresenter.eliminarPedidoById`).

Última actualización: 2026-03-16

0. Resumen Ejecutivo

Migrar un evento de impresión a PrinterServices requiere cambios en **4 capas**:

#	Capa	Archivo(s) típicos	Qué hacer
1	Backend Strategy	ServerAnularCorregirStrategy.cs (nuevo)	Crear estrategia que envíe a PS vía PrinterServiceClient
2	Backend Controller	PedidoController.cs	Interceptar con flag → delegar a ServerImpresionService → inyectar PrintJobResults
3	Backend Trazabilidad	PedidoprinterjobController.cs	Persistir pedidoprinterjob con TIPO, PEDIDOID, VENTAID, DELIVERYID, MODALIDAD
4	Frontend Presenter	FragmentDetallePedidoPresenter.cs	Detectar flag → mostrar PSProgressService (centralizado)

Regla de oro: El **servidor decide**, el **Front confía**. Si `ImpresionList` viene vacía + flag ON = PrinterServices ya está imprimiendo.

1. Arquitectura de Estrategias en Backend

1.1 Estrategias existentes

Las estrategias de impresión ya viven en el backend (`QuipuNetX/Services/Print/Strategies/`). Cada tipo de documento tiene su propia estrategia que sabe qué documentos enviar a PrinterServices:

Strategy	Tipo	Documentos que imprime	Estado
ServerComandasStrategy	Comandas	Comandas (excluye stickers Jaltech)	✅ Implementada
ServerVentaRapidaStrategy	VentaRapida	Comprobante + Comandas + Sorteo + Encuesta + Motorizado	✅ Implementada
ServerVentaSalonStrategy	VentaSalon	Comprobante + Sorteo + Encuesta	✅ Implementada
ServerDeliveryStrategy	VentaDelivery	Comprobante + Sorteo + Encuesta + Motorizado	✅ Implementada
ServerAnularCorregirStrategy	VentaAnularCorregir	Comanda de anulación	⬜ Pendiente (Fase 7C)

ServerVentaProveedorStrategy	VentaProveedor	Factura electrónica	Pendiente (Fase 7C)
------------------------------	----------------	---------------------	------------------------------------

1.2 DOS patrones coexistentes (importante)

En el código actual hay **dos formas** en que los Controllers envían impresiones a PrinterServices:

Patrón A — Directo (Fase 7A, PedidoController.addLista)

```
PedidoController.addLista
  → PrinterServiceClient.Instance.EnviarComandasAsync()      ← DIRECTO
  → PedidoprinterjobController.guardarJobsPorComandas()      ← método específico
  → PrintJobStatusManager.Instance.RegisterJobs()             ← manual
  → Fix race condition BD (foreach → actualizarEstadoJob)    ← manual
```

Quién lo usa: PedidoController.addLista (enviar pedidos a cocina).

Por qué es directo: Fue la primera migración (Fase 7A), cuando todavía no existía la arquitectura Strategy. El Controller llama a PrinterServiceClient directamente, hace la persistencia, el registro en RAM, y el fix de race condition, todo manualmente.

Patrón B — Strategy (Fase 7B, VentaController, DeliveryController)

```
VentaController / DeliveryController
  → ImpresionContext.CrearParaVentaRapida/Delivery/Salon()
  → ServerImpresionService.Instance.ImprimirAsync(ctxPS)
  → ImpresionStrategyFactory.CrearEstrategia(context)
  → switch (context.TipoImpresion)
    → ServerVentaRapidaStrategy / ServerDeliveryStrategy / etc.
      → PrinterServiceClient + guardarJobsPorTipo + RegisterJobs (todo interno)
      → HTTP POST a PrinterServices :8090
```

Quién lo usa: VentaController.addVentaRapidaMovil , VentaController.addVentaDeliveryMovil , DeliveryController.generarVentaEImpresionDelivery .

Ventaja: La Strategy encapsula todo internamente — el Controller solo crea el contexto, llama ImprimirAsync , y evalúa el resultado.

¿Cuál usar para nuevas migraciones?

Caso	Patrón recomendado	Razón
Solo comandas (tipo COMANDA)	Patrón A (Directo)	Ya existe ServerComandasStrategy pero addLista no la usa. Para anulaciones es más simple usar el patrón directo porque la Strategy ServerComandasStrategy fue diseñada para el mismo caso.
Ventas con múltiples documentos	Patrón B (Strategy)	La Strategy maneja automáticamente comprobante + sorteo + encuesta + motorizado + comandas y su persistencia
Documento nuevo complejo	Patrón B (Strategy)	Crear nueva Strategy para encapsular toda la lógica

1.3 Ejemplo de migración: Anulación de Pedidos usa Patrón A (Directo)

¿Por qué NO usar VentaAnularCorregir Strategy?

La Strategy `VentaAnularCorregir` está **pendiente (Fase 7C)** y comentada en la factory. Además, `ImpresionContext.EsValido()` para `VentaAnularCorregir` **requiere `VentaId`**, pero una anulación de pedido en modo salón **no tiene `venta_id`** — solo tiene `pedido_id`.

Una anulación de pedido genera **una comanda de anulación** (tipo `COMANDA`), no un comprobante de venta. Por lo tanto, el patrón correcto es el **Patrón A (Directo)**, copiando exactamente el flujo de `PedidoController.addLista`.

El código completo del Patrón A se detalla en la **§6 (Backend Controller)**.

1.4 ¿Cuándo sí crear una nueva Strategy?

Crear nueva Strategy solo cuando:

- El tipo de impresión genera **múltiples documentos** (comprobante + sorteo + encuesta + etc.)
- Se necesita un `TipoImpresionOrigen` nuevo en `ImpresionContext`
- El flujo es lo suficientemente diferente como para no reusar las strategies existentes

Para crear una nueva Strategy, el proceso es:

1. Crear archivo en `QuipuNetX/Services/Print/Strategies/Server[Tipo]Strategy.cs` implementando `IPrintStrategy`
2. Registrar en `ImpresionStrategyFactory.cs` (descomentar/agregar case en switch)
3. Agregar factory method en `ImpresionContext.cs` (ej: `CrearPara[Tipo]()`)
4. Ajustar `EsValido()` si los requisitos de validación son diferentes

1.5 DRY: Método `ExtraerYRegistrarJobs` duplicado

Actualmente `ExtraerYRegistrarJobs` está copiado en `ServerDeliveryStrategy` y `ServerVentaRapidaStrategy`. Para nuevas strategies, extraer a una clase base o helper estático:

```
// QupuNetX/Services/Print/Strategies/StrategyHelper.cs
public static class StrategyHelper
{
    public static void ExtraerYRegistrarJobs(Respuesta resp, string tipo, string ventaId,
        List<PrintJobResult> todosLosJobs, string modalidad = null, string deliveryId =
null)
    {
        // ... (misma lógica que ServerDeliveryStrategy.ExtraerYRegistrarJobs)
    }

    public static string ObtenerMensajeError(Respuesta resp)
    {
        if (resp.Mensajes != null && resp.Mensajes.Count > 0) return resp.Mensajes[0];
        return "Error desconocido";
    }
}
```

2. Tabla `pedidoprinterjob` — Trazabilidad Bidireccional

2.1 Estructura de la tabla

```
CREATE TABLE pedidoprinterjob (
    ID                INTEGER PRIMARY KEY AUTOINCREMENT,
    PEDIDOID          TEXT,          -- ID del pedido (null si tipo ≠ COMANDA)
    VENTAID           TEXT,          -- ID de la venta (null si tipo = COMANDA sin venta)
```

```

TIPO          TEXT NOT NULL, -- Tipo de documento impreso
JOBID         TEXT NOT NULL, -- UUID del job en PrinterServices
STATUS        TEXT NOT NULL, -- SENT → DONE | FAILED | EXPIRED | CANCELLED
FECHACREACION TEXT NOT NULL, -- yyyy-MM-dd HH:mm:ss
MODALIDAD     TEXT,          -- Modalidad de origen (1=Salon, 2=Rapida, etc.)
DELIVERYID    TEXT          -- ID del delivery asociado (null si no aplica)
);

```

2.2 Mapeo: Tipo de documento → Campos en pedidoprinterjob

Cada tipo de documento se vincula a entidades de negocio **distintas**. Esta es la regla de vinculación:

Acción de negocio	TIPO	PEDIDOID	VENTAID	DELIVERYID	MODALIDAD	Ejemplo
Enviar pedidos a cocina (salón)	COMANDA	870936	NULL	NULL	1 (Salon)	Comanda de cocina en mesa
Enviar pedidos a cocina (delivery)	COMANDA	870936	NULL	DEL-001	3 (Delivery)	Comanda de delivery
Enviar pedidos (venta rápida)	COMANDA	870941	NULL	NULL	2 (Rapida)	Comanda de venta rápida
Cobrar venta (salón)	COMPROBANTE	NULL	VTA-100	NULL	1 (Salon)	Boleta/factura salón
Cobrar venta (rápida)	COMPROBANTE	NULL	VTA-101	NULL	2 (Rapida)	Boleta/factura rápida
Cobrar venta (delivery)	COMPROBANTE	NULL	VTA-102	DEL-001	3 (Delivery)	Boleta/factura delivery
Sorteo/Promoción	PROMOCION	NULL	VTA-100	NULL	1	Ticket de sorteo
Encuesta QR	ENCUESTA	NULL	VTA-100	NULL	1	Ticket con Q encuesta
Precuenta motorizado	PRECuenta	NULL	VTA-102	DEL-001	3	Despacho para motorizado
Despacho delivery	DELIVERY	NULL	VTA-102	DEL-001	3	Copia para motorizado
Anular pedido (salón)	COMANDA	870936	NULL	NULL	1 (Salon)	Comanda de anulación
Anular pedido (venta rápida)	COMANDA	870941	VTA-103	NULL	2 (Rapida)	Anulación cc ventald
Reimprimir comanda	COMANDA	870936	NULL	NULL	1	Reimpresión de comanda

2.3 Regla de vinculación por tipo

COMANDA:

- PEDIDOID = pedido_id (SIEMPRE, 1 registro por pedido del job)
- VENTAID = null (excepto en venta rápida donde la venta se genera a la vez)
- DELIVERYID = delivery_id (si modalidad es delivery)

COMPROBANTE / PROMOCION / ENCUESTA / PRECUENTA / DELIVERY:

- PEDIDOID = null (no se vincula a pedido individual)
- VENTAID = venta_id (SIEMPRE)
- DELIVERYID = delivery_id (si modalidad es delivery)

2.4 Trazabilidad bidireccional: ¿Por qué importa?

La tabla `pedidoprinterjob` permite responder dos preguntas críticas:

1. **Job → Negocio:** "El job `3c74c9ba0b6f` falló. ¿Qué pedidos/ventas/deliverys se ven afectados?"

```
SELECT PEDIDOID, VENTAID, DELIVERYID FROM pedidoprinterjob WHERE JOBID = '3c74c9ba0b6f'
```

2. **Negocio → Job:** "El pedido `870936` ¿se imprimió? ¿Cuál fue el resultado?"

```
SELECT JOBID, STATUS, TIPO FROM pedidoprinterjob WHERE PEDIDOID = '870936' ORDER BY FECHACREACION DESC
```

Esto alimenta los **badges visuales** en el Front:

- `Pedido.IsPrinterComandaFailed` → `brComandaFailed` en `ListItemPedido.xaml`
- `Delivery.IsPrinterComandaFailed` → Badge en lista de deliverys
- `brdIconoImpresora` (global) → Cualquier fallo, independiente del tipo

2.5 guardarJobsPorComandas vs guardarJobsPorTipo

Método	Usado por	Cuándo usar
<code>guardarJobsPorComandas()</code>	<code>PedidoController.addLista</code> (comandas directas)	Cuando el Controller llama directamente a <code>PrinterServiceClient</code> SIN pasar por Strategy
<code>guardarJobsPorTipo()</code>	Strategies (dentro de <code>ExtraerYRegistrarJobs</code>)	Cuando se usa la arquitectura Strategy (método preferido para nuevas migraciones)

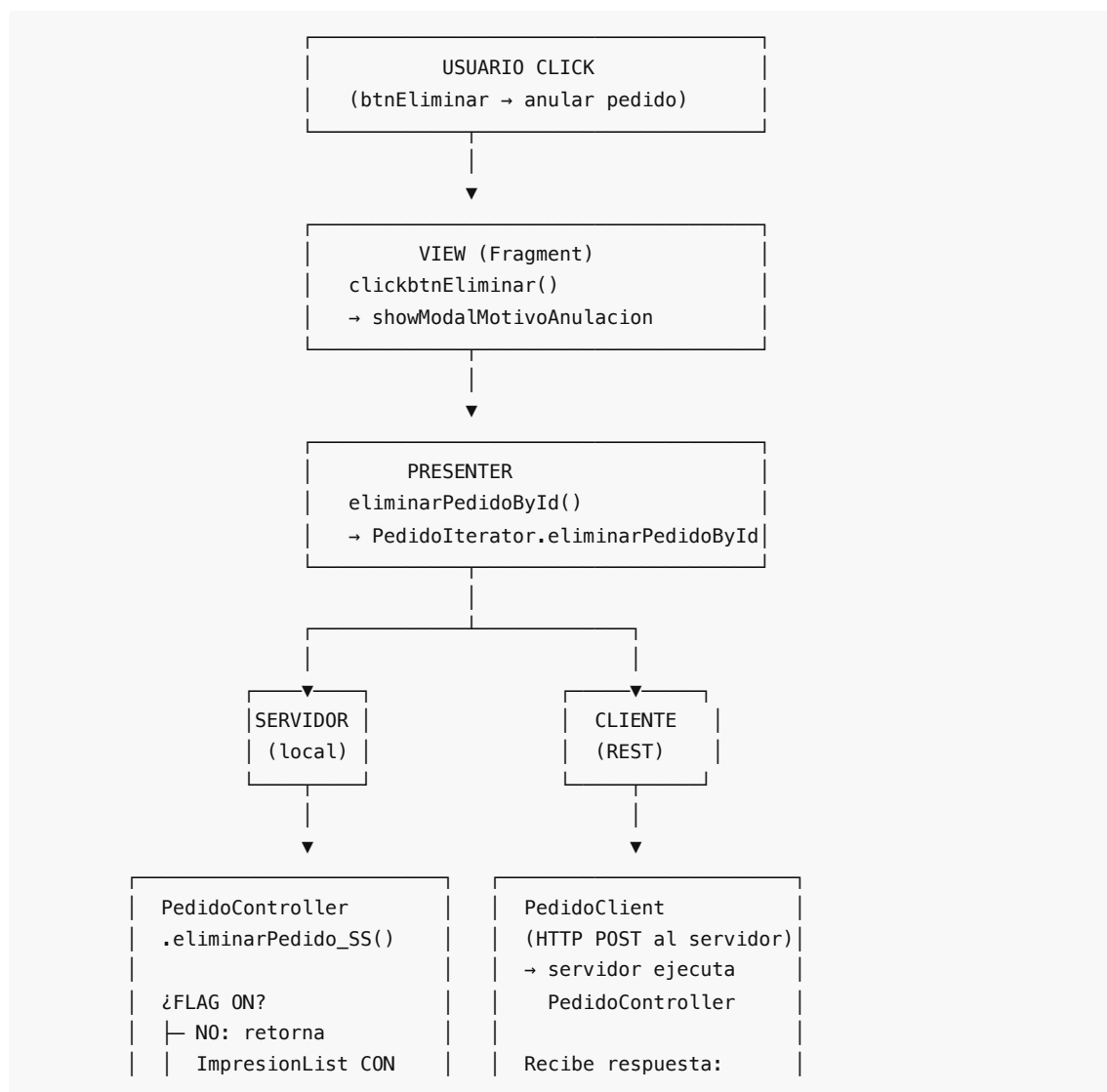
Para nuevas migraciones: Siempre usar **Strategy + `guardarJobsPorTipo()`** (dentro de `ExtraerYRegistrarJobs`). El Controller NO debe llamar a `PrinterServiceClient` directamente.

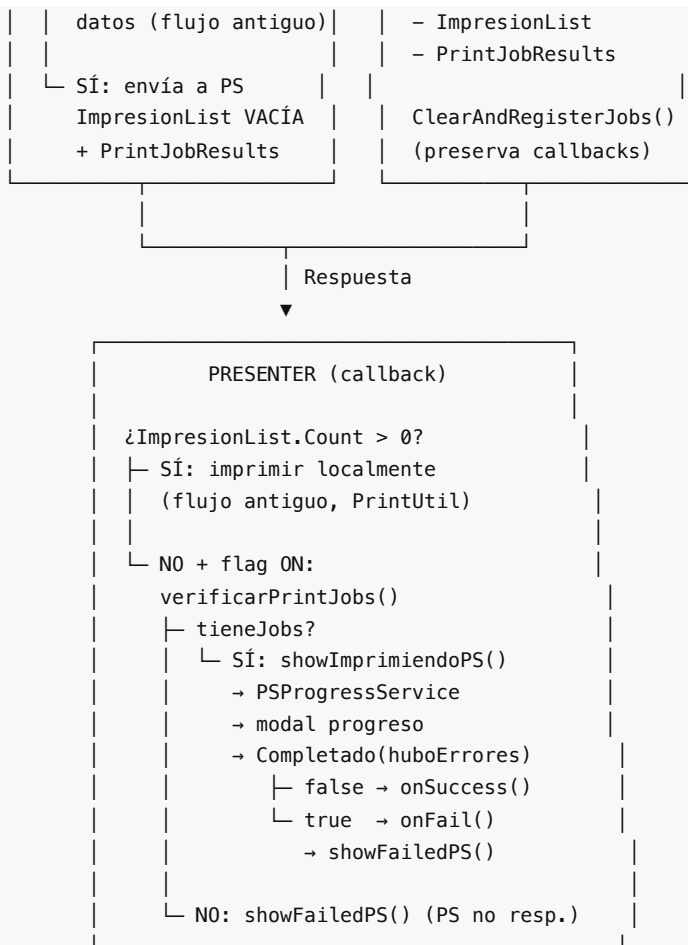
3. Tabla de Decisión (4 escenarios)

Terminal	Flag Servidor	Qué pasa
SERVIDOR, flag OFF	USAR_PRINTER_SERVICE=0	Controller retorna <code>ImpresionList</code> CON datos
		→ Front imprime localmente (flujo antiguo)

SERVIDOR, flag ON	USAR_PRINTER_SERVICE=1	Controller envía a PS, retorna ImpresionList
		VACÍA + PrintJobResults → Front monitorea
CLIENTE, flag OFF	Servidor remoto OFF	REST → servidor retorna ImpresionList CON
		datos → Front imprime localmente
CLIENTE, flag ON	Servidor remoto ON	REST → servidor envía a PS, retorna
		ImpresionList VACÍA + PrintJobResults
		→ PedidoClient parsea → Front monitorea

2. Diagrama de Flujo Completo





5. Checklist de Migración

Usa esta checklist para cada evento de impresión que migres:

BACKEND (QuipuNetX)

- ☐ **Paso B1:** Crear Strategy en `QuipuNetX/Services/Print/Strategies/` (ver §1.3)
- ☐ **Paso B2:** Registrar en `ImpresionStrategyFactory` (ver §1.4)
- ☐ **Paso B3:** Agregar factory method en `ImpresionContext` (ver §1.5)
- ☐ **Paso B4:** En el Controller: flag guard → crear `ImpresionContext` → `ServerImpresionService.ImprimirAsync()` → vaciar `ImpresionList`
- ☐ **Paso B5:** Inyectar `PrintJobResults` en `Respuesta` para el Front
- ☐ **Paso B6:** Manejar PS no responde → `PrintJobResults` vacío → Front muestra error claro

FRONTEND (QuipuNet)

- ☐ **Paso F1:** En el Presenter, detectar `ImpresionList` vacía + flag ON
- ☐ **Paso F2:** Llamar `verificarPrintJobs()` → `showImprimiendoPrinterService` (ya existe en la View)
- ☐ **Paso F3:** Si la View NO tiene `showImprimiendoPrinterService`, implementar usando `PrinterServiceUIHelper` (ver §8)
- ☐ **Paso F4:** (Opcional) Badge a nivel de item (`IsPrinterComandaFailed` o equivalente)

- ☐ **Paso F5:** (Solo si aplica modo cliente) Verificar que el Client parsea `printJobResults`

6. BACKEND Controller — Paso a Paso con Ejemplo de Anulación

6.1 Patrón usado: Directo (Patrón A)

La anulación de pedidos genera una **comanda de anulación** (tipo `COMANDA`). No tiene `VentaId`, solo `pedido_id`. Por eso usa el **Patrón A (Directo)** — el mismo que `PedidoController.addLista`.

Referencia real: `PedidoController.addLista` líneas 650-728 en `QuipuNetX/controller/PedidoController.cs`

6.2 Paso B4: Flag Guard + Envío directo a PrinterServiceClient

Archivo: `QuipuNetX/controller/PedidoController.cs` (método `eliminarPedido_SS` o equivalente)

```
// DESPUÉS de generar resultadoComandasPreimpresion (ImpresionList de anulación):
// -----
// Fase PS: Si flag ON + modo servidor → enviar a PS directamente (Patrón A)
// Copiado de PedidoController.addLista (líneas 650-728)
// -----
List<PrintJobResult> printJobResultsAnulacion = null;

if (Util.esModoServidor()
    && FeatureFlagConfigReader.IsEnabled("USAR_PRINTER_SERVICE")
    && resultadoComandasPreimpresion != null
    && resultadoComandasPreimpresion.Count > 0)
{
    // 1. Copiar lista antes de vaciar
    var comandasParaImprimir = new List<Impresion>(resultadoComandasPreimpresion);
    var pedidoIds = new List<string>();
    if (!string.IsNullOrEmpty(pedido_id)) pedidoIds.Add(pedido_id);

    // 2. Vaciar ImpresionList para que Front NO imprima localmente
    resultadoComandasPreimpresion = new List<Impresion>();

    // 3. Limpiar jobs de sesiones anteriores ANTES del POST a PS
    //     IMPORTANTE: Clear() ANTES del POST. Si está después, borra jobs FAILED
    //     que el callback ya puso (PS procesa en <5ms con SemaphoreSlim)
    PrintJobStatusManager.Instance.Clear();

    // 4. Enviar a PrinterServices DIRECTAMENTE (Patrón A)
    Respuesta respuestaPS = null;
    try
    {
        respuestaPS = System.Threading.Tasks.Task.Run(async () =>
        {
            return await PrinterServiceClient.Instance.EnviaComandasAsync(
                comandasParaImprimir, pedidoIds, "Imprimiendo anulación...",
                ipClienteRest);
        }).GetAwaiter().GetResult();
    }
    catch (Exception ex)
    {
        Util.Capture(ex, "PedidoController", "eliminarPedido.PrinterService", ex.StackTrace);
    }
}
```



```

?? "-");
}

// 5. Evaluar respuesta y guardar trazabilidad
if (respuestaPS != null && respuestaPS.Tipo == Util.SUCCESS
    && respuestaPS.Data is List<PrintJobResult> jobResults && jobResults.Count > 0)
{
    // 5a. Persistir relación pedido↔job en BD (tabla pedidoprinterjob)
    PedidoprinterjobController.guardarJobsPorComandas(jobResults, modalidadVenta,
delivery_id);

    // 5b. Registrar jobs en RAM para callbacks en tiempo real
    // RegisterJobs NO sobrescribe jobs que ya existen (fix race condition)
    PrintJobStatusManager.Instance.RegisterJobs(jobResults);

    // 5c. Fix race condition BD: si callback llegó antes del INSERT
    // El callback hizo UPDATE WHERE JOBID=? → 0 rows (no existía aún)
    // Ahora el registro SÍ existe → re-ejecutar UPDATE
    foreach (var jr in jobResults)
    {
        var jobInfo = PrintJobStatusManager.Instance.GetJobStatus(jr.JobId);
        if (jobInfo != null && (jobInfo.Status == "FAILED" || jobInfo.Status ==
"EXPIRED"))
        {
            PedidoprinterjobController.actualizarEstadoJob(jr.JobId, jobInfo.Status);
        }
    }

    // 5d. Guardar para inyectar en respuesta al Front
    printJobResultsAnulacion = jobResults;
}
// Si PS no respondió: printJobResultsAnulacion queda null
// → El Front recibirá PrintJobResults vacío → mostrará "Servicio de impresión no
responde"
}

```

6.3 Paso B5: Inyectar PrintJobResults en la Respuesta

Al final del método, donde se construye la Respuesta :

```

var respuestaFinal = new Respuesta(tipo, data, mensajes, resultadoComandasPreimpresion, 1);
if (printJobResultsAnulacion != null)
{
    respuestaFinal.PrintJobResults = printJobResultsAnulacion;
}
return respuestaFinal;

```

6.4 Paso B6: PS no responde — Qué pasa

Cuando PrinterServices no responde (timeout, apagado, red caída):

```

SERVIDOR (Patrón A):
PrinterServiceClient.Instance.EnviaComandasAsync()
→ PostConReintentoAsync: 2 intentos × 10s timeout = 20s máximo
→ respuestaPS = null (excepción capturada) o Tipo=ERROR

```

```
→ printJobResultsAnulacion = null (NO hay jobs)
→ Respuesta al Front: ImpresionList vacía + PrintJobResults null
```

FRONT:

```
→ ImpresionList vacía + flag ON → verificarPrintJobs()
→ tieneJobs = false (PrintJobResults es null o vacío)
→ showErrorRegisterView("Servicio de impresión no responde. Verifique que PrinterServices
esté activo.")
```

No se usa `showFailedPrinterService` en este caso porque NO hay jobs que monitorear. Se muestra un error claro y directo al usuario.

6.5 Comparación: Patrón A vs Patrón B en Controller

Aspecto	Patrón A (Directo)	Patrón B (Strategy)
Quién llama a PS	Controller directamente	Strategy internamente
Persistencia	Controller llama guardarJobsPorComandas	Strategy llama guardarJobsPorTipo vía ExtraerYRegistrarJobs
Registro RAM	Controller llama RegisterJobs	Strategy lo hace internamente
Fix race condition BD	Controller hace el foreach manual	Strategy lo hace internamente
Líneas en Controller	~50 líneas	~20 líneas (solo contexto + ImprimirAsync + evaluar resultado)
Ejemplo real	PedidoController.addLista:650-728	VentaController.addVentaRapidaMovil:1580-1644

7. FRONTEND Presenter — Paso a Paso

Paso F1: Detectar ImpresionList vacía + flag ON

Archivo: `front/QuipuNet/MainApp/venta/Pedidos/FragmentDetallePedidoPresenter.cs`

Método: `eliminarPedidoById` (línea 489)

El código actual (línea 506-522) usa `PrinterQueueManager` o `PrintUtil.imprimirComandasEthernet`. Ese código **sigue siendo válido para cuando el flag está OFF** (flujo antiguo). Solo se agrega un `else if` para el caso flag ON:

```
bool configuracionDeshabilitarImpresion = Configuracion.estaActivado(
    Definitions.CONFIGURACION_DESHABILITAR_IMPRESION_AL_ANULAR_O_MODIFICAR_PEDIDOS);

if (!configuracionDeshabilitarImpresion)
{
    if (respuesta.ImpresionList?.Count > 0)
    {
        // =====
        // FLAG OFF (o servidor sin PS): ImpresionList tiene datos
        // → Imprimir localmente con el flujo antiguo
        // NOTA: Este bloque NO se toca. Es el flujo existente que sigue
        // funcionando cuando USAR_PRINTER_SERVICE está desactivado.
        // =====
    }
}
```

```

        if
        (Util.estaConfiguracionActivadaPOS3(Definitions.CONFIGURACIONPOS_UTILIZAR_COLA_IMPRESIONES))
        {
            foreach (Impresion impresion in respuesta.ImpresionList)
            {
                PrinterQueueManager.Instance.Enqueue(new PrintComandaCommand(impresion));
            }
        }
        else
        {
            PrintUtil.imprimirComandasEthernet(respuesta.ImpresionList, respuestaPrint =>
            {
                if (_disposed || _cts?.IsCancellationRequested == true) return;
            });
        }
    }
}
// =====
// FLAG ON: ImpresionList vacía → PrinterServices ya está imprimiendo
// Aplica tanto en modo servidor (flag local) como modo cliente
// (el servidor remoto tiene el flag activo).
// =====
else if (FeatureFlagManager.Instance.IsEnabled("USAR_PRINTER_SERVICE")
    || (Util.esModoCliente() && UtilFront.FEATURE_FLAG_FROM_SERVER_PRINTER_SERVICES))
{
    this.verificarPrintJobsAnulacion(respuesta);
}
}

```

Paso F2: Método verificarPrintJobs

Patrón ya existente en `FragmentDetallePedidoPresenter.verificarPrintJobsPorComandas` (línea 437). Para anulación, crear método similar:

```

private void verificarPrintJobsAnulacion(Respuesta respuesta)
{
    try
    {
        if (_disposed || _cts?.IsCancellationRequested == true ||
            fragmentDetallePedidoIView == null)
            return;

        bool tieneJobs = respuesta.PrintJobResults != null &&
            respuesta.PrintJobResults.Count > 0;

        if (tieneJobs)
        {
            // PrinterServices registró los jobs – mostrar modal de progreso
            fragmentDetallePedidoIView.showImprimiendoPrinterService(
                onSuccess: () =>
                {
                    // Todos los jobs OK – pedido ya se eliminó, nada adicional
                },
                onFail: () =>
                {
                    // Al menos un job falló – mostrar modal de error con reintento
                }
            );
        }
    }
}

```

```

        fragmentDetallePedidoIView.showFailedPrinterService();
    },
    cantidadJobs: respuesta.PrintJobResults.Count
);
}
else
{
    // PS no respondió o no generó job_ids → error claro al usuario
    fragmentDetallePedidoIView.showErrorRegisterView(
        "Servicio de impresión no responde. Verifique que PrinterServices esté
activo.");
}
}
catch (Exception ex)
{
    NLog.LogManager.GetCurrentClassLogger().Error(ex, "Error verificando
PrintJobResults anulación");
    fragmentDetallePedidoIView.showErrorRegisterView(
        "Servicio de impresión no responde. Verifique que PrinterServices esté
activo.");
}
}
}

```

8. Centralización de showImprimiendoPrinterService

8.1 El problema actual: código duplicado en 5+ Views

Actualmente `showImprimiendoPrinterService` está copiado idénticamente en:

- `ListaPedidosTemporales.cs`
- `FragmentDetallePedido.cs`
- `DeliverysConfirmadosNew.cs`
- `ListaDeliveryPendiente.cs`
- `ResumenDelivery.cs`

Cada View tiene **el mismo código** (~60 líneas) para crear `PSProgressService`, suscribirse a `Completado`, mostrar/cerrar el modal, etc.

8.2 ¿Por qué NO se puede hacer PSProgressService singleton/global?

`PSProgressService` **NO es ni debe ser singleton**. Razones:

1. **Tiene estado de batch:** `_cantidadTotal`, `_cantidadCompletados`, `_hayErroresEnCola` son específicos de cada operación de impresión.
2. **Tiene ciclo de vida finito:** Se crea al iniciar una operación de impresión y se destruye (desuscribe) cuando termina.
3. **Puede haber múltiples operaciones:** En teoría, dos terminales podrían generar operaciones de impresión overlapping.

Sin embargo, `PrintJobStatusManager` **SÍ es singleton** y ya maneja internamente la diferencia entre modo cliente y servidor. Por eso `PSProgressService` funciona en ambos modos sin necesidad de que cada View sepa en qué modo está.

8.3 Solución: PrinterServiceUIHelper (helper estático centralizado)

En vez de copiar el código en cada View, extraer a un helper estático:

```

// QuipuNet/MainApp/Services/PrinterServiceUIHelper.cs
using System;
using System.Windows;

namespace QuipuNet.MainApp.Services
{
    /// <summary>
    /// Helper centralizado para mostrar modales de PrinterServices.
    /// Elimina la necesidad de copiar showImprimiendoPrinterService en cada View.
    /// </summary>
    public static class PrinterServiceUIHelper
    {
        /// <summary>
        /// Muestra modal de progreso de PrinterServices y ejecuta callbacks al finalizar.
        /// Reemplaza las 5+ copias de showImprimiendoPrinterService en las Views.
        /// </summary>
        public static void MostrarProgresoImpresion(Action onSuccess, Action onFail, int
cantidadJobs = 0)
        {
            Application.Current?.Dispatcher?.BeginInvoke(new Action(() =>
            {
                try
                {
                    var psProgress = new PSProgressService(cantidadJobs);

                    // Race condition: jobs ya terminaron antes de crear PSProgressService
                    if (psProgress.YaFinalizado)
                    {
                        psProgress.Detener(); // Limpiar suscripciones
                        if (psProgress.YaFinalizadoConErrores)
                            onFail?.Invoke();
                        else
                            onSuccess?.Invoke();
                        return;
                    }

                    var modalImprimiendo = new CustomModalImprimiendoNewView(psProgress);

                    psProgress.Completado += (huboErrores) =>
                    {
                        try
                        {
                            if (modalImprimiendo != null && modalImprimiendo.IsLoaded)
                                modalImprimiendo.Close();

                            var transicionTimer = new
System.Windows.Threading.DispatcherTimer
                            {
                                Interval = TimeSpan.FromMilliseconds(500)
                            };
                            transicionTimer.Tick += (s, e2) =>
                            {
                                transicionTimer.Stop();
                                if (huboErrores) onFail?.Invoke();
                                else onSuccess?.Invoke();
                            }
                        }
                    }
                }
            })
            );
        }
    }
}

```

```

        };
        transicionTimer.Start();
    }
    catch (Exception ex)
    {
        NLog.LogManager.GetCurrentClassLogger().Error(ex,
            "Error en Completado de PSPProgressService");
    }
};

modalImprimiendo.Show();
var desktopWorkingArea = SystemParameters.WorkArea;
modalImprimiendo.Left = desktopWorkingArea.Right -
modalImprimiendo.Width - 40;
modalImprimiendo.Top = desktopWorkingArea.Bottom -
modalImprimiendo.Height - 25;
}
catch (Exception ex)
{
    NLog.LogManager.GetCurrentClassLogger().Error(ex,
        "PrinterServiceUIHelper.MostrarProgresoImpresion EXCEPCIÓN");
}
}));
}

/// <summary>
/// Muestra modal de impresión fallida con opción de reintento.
/// </summary>
public static void MostrarErrorImpresion()
{
    Application.Current?.Dispatcher?.BeginInvoke(new Action(() =>
    {
        try
        {
            if (!CustomModalImpresionFallidaNew.PuedeAbrir()) return;

            var viewModel = new PSModaImpresionFallidaViewModel();
            var modalFallida = new CustomModalImpresionFallidaNew(viewModel);
            modalFallida.Show();

            var desktopWorkingArea = SystemParameters.WorkArea;
            modalFallida.Left = desktopWorkingArea.Right - modalFallida.Width - 40;
            modalFallida.Top = desktopWorkingArea.Bottom - modalFallida.Height -
25;

        }
        catch (Exception ex)
        {
            NLog.LogManager.GetCurrentClassLogger().Error(ex,
                "PrinterServiceUIHelper.MostrarErrorImpresion EXCEPCIÓN");
        }
    }));
}
}
}
}

```

8.4 Uso desde cualquier Presenter (sin tocar la View)

Con el helper centralizado, el Presenter puede llamar directamente sin necesidad de que la View implemente métodos:

```
// En CUALQUIER presenter, sin necesidad de interfaz IView:
private void verificarPrintJobsAnulacion(Respuesta respuesta)
{
    bool tieneJobs = respuesta.PrintJobResults != null && respuesta.PrintJobResults.Count > 0;

    if (tieneJobs)
    {
        PrinterServiceUIHelper.MostrarProgresoImpresion(
            onSuccess: () => { /* nada adicional */ },
            onFail: () => { PrinterServiceUIHelper.MostrarErrorImpresion(); },
            cantidadJobs: respuesta.PrintJobResults.Count
        );
    }
    else
    {
        // PS no respondió – mostrar error claro
        fragmentDetallePedidoIView?.showErrorRegisterView(
            "Servicio de impresión no responde. Verifique que PrinterServices esté activo.");
    }
}
```

8.5 Migración gradual

No es necesario refactorizar las 5 Views existentes de golpe. Se puede:

1. **Nuevas migraciones:** Usar `PrinterServiceUIHelper` directamente desde el Presenter
2. **Views existentes:** Refactorizar gradualmente, reemplazando el código duplicado por llamadas al helper
3. **Interfaces IView:** Los métodos `showImprimiendoPrinterService` / `showFailedPrinterService` en las interfaces pueden delegarse al helper internamente

9. Modo CLIENTE — Consideraciones Especiales

9.1 PedidoClient ya parsea PrintJobResults

Si la migración es para un método que ya pasa por `PedidoClient` (como `eliminarPedidoById`), verificar que el Client correspondiente parsee `printJobResults` de la respuesta JSON.

Patrón existente en `PedidoClient.cs` (línea 286-321):

```
if (Util.esModoCliente()
    && PrintJobStatusManager.Instance.IsClientMode
    && response.ContainsKey("printJobResults")
    && (response["printJobResults"] as JToken).Type != JTokenType.Null)
{
    // Parsear printJobResults del JSON
    var printJobResultsArray = response["printJobResults"] as JArray;
    foreach (var item in printJobResultsArray) { ... }
```

```
// Registrar jobs con método atómico (preserva callbacks de PS)
PrintJobStatusManager.Instance.ClearAndRegisterJobs(printJobResults);
}
```

Si el método NO pasa por **PedidoClient** (ej: usa otro Client), debes agregar este parseo en el Client correspondiente.

9.2 ClearAndRegisterJobs vs Clear+RegisterJobs

SIEMPRE usar **ClearAndRegisterJobs()** en modo cliente. Nunca **Clear()** + **RegisterJobs()** separados.

Razón: En modo cliente, el callback de PS llega por puerto 8083 ANTES que la respuesta REST. **Clear()** destruiría el estado FAILED que el callback ya puso. **ClearAndRegisterJobs()** preserva los jobs del batch actual que ya recibieron callback.

9.3 Timeline comparativa

SERVIDOR (funciona con **Clear()** ANTES del POST):

T1: **Clear()** → vacío
T2: **POST** a PS → PS procesa
T3: **Callback** → **UpdateStatus(FAILED)** → job creado
T4: **RegisterJobs** → job ya existe → preservado ✓
T5: **PSProgressService** → ve **FAILED** → finaliza ✓

CLIENTE (requiere **ClearAndRegisterJobs()**):

T1: **POST** vía REST → servidor → PS procesa
T2: **Callback :8083** → **UpdateStatus(FAILED)** → job creado
T3: **REST response** → **ClearAndRegisterJobs** → job preservado ✓
T4: **PSProgressService** → ve **FAILED** → finaliza ✓

10. Badge a Nivel de Item (Opcional por tipo)

10.1 Tabla de vinculación tipo → item

Cada tipo de impresión se vincula a un item específico en la UI:

Tipo de impresión	Se vincula a	Propiedad del item	Badge XAML	Pantalla
Comandas	Pedido	<code>Pedido.IsPrinterComandaFailed</code>	<code>brComandaFailed</code> en <code>ListItemPedido.xaml</code>	Lista pedidos de mesa
Precuentas	Mesa	<code>Mesa.IsPrinterFailed</code> (crear)	Badge en item mesa	Lista mesas
Deliverys	Delivery	<code>Delivery.IsPrinterComandaFailed</code>	Badge en item delivery	Lista deliverys
Ventas	Venta	<code>Venta.IsPrinterFailed</code> (crear)	Badge en item venta	Historial ventas
Anulación	Pedido	<code>Pedido.IsPrinterComandaFailed</code>	<code>brComandaFailed</code> en <code>ListItemPedido.xaml</code>	Lista pedidos de mesa

10.2 Cómo funciona el badge a nivel de item (patrón existente)


```
PrintJobStatusManager.AnyJobFailed(jobId, error)
→ ListaPedidos.OnComandaFailed(jobId, error)
→ Busca pedidoIds del job
→ Busca pedidos en la lista visual
→ pedido.IsPrinterComandaFailed = true ← DataTrigger en XAML activa animación
```

Para **anulación de pedidos**, el `IsPrinterComandaFailed` ya existe en `Pedido` y el XAML ya tiene el `DataTrigger`. Solo necesitas que `ListaPedidos.OnComandaFailed` sepa vincular el jobId al pedidoId correcto.

10.3 vs Badge global (brdIconoImpresora)

El badge **global** (`brdIconoImpresora` en `UtilesCabecera_New.xaml`) es **independiente** del tipo y **siempre se activa** cuando hay CUALQUIER fallo:

```
PrintJobStatusManager.AnyJobFailed
→ UtilesCabeceraViewModel.OnPrintJobFailed
→ ImpresionesFallidas.Add(...)
→ iniciarAnimacion()
→ brdIconoImpresora parpadea rojo ← SIEMPRE, independiente del tipo
```

No necesitas hacer nada adicional para el badge global. Ya se activa automáticamente para cualquier tipo de impresión que pase por `PrintJobStatusManager.AnyJobFailed`.

11. Notificación Global — Flujo de PSProgressService

`PSProgressService` es quien orquesta el modal de progreso y dispara los callbacks `onSuccess` / `onFail` :

```
PSProgressService

Constructor(cantidadJobs):
1. Se suscribe a PrintJobStatusManager:
  - JobStatusChanged → OnJobStatusChanged (actualiza progreso)
  - AllJobsDone      → OnAllJobsDone (finaliza)
  - AnyJobFailed     → OnAnyJobFailed (barra roja)

2. Check inmediato (catch-up para race conditions):
  - GetAllJobs() → si todos terminales → YaFinalizado=true
  - El caller verifica YaFinalizado ANTES de mostrar modal

Eventos que recibe:
JobStatusChanged → actualiza barra de progreso
AnyJobFailed     → cambia barra a rojo
AllJobsDone      → dispara Completado(huboErrores) después de
                  800ms delay (para que usuario vea 100%)

Completado(huboErrores):
→ Caller cierra modal de progreso
→ Si huboErrores → onFail() → showFailedPrinterService()
→ Si !huboErrores → onSuccess() → navegación normal

Timeout:
- Servidor: 30s (PS local, callbacks <5ms)
- Cliente: 300s (callbacks por red, PS puede tardar minutos)
```

12. Ejemplo Completo: Migración de Anulación de Pedidos

12.1 Flujo ANTES (sin PrinterServices)

```
btnEliminar_Click
→ FragmentDetallePedido.clickbtnEliminar()
→ showModalMotivoAnulacion(usuarioId)
→ FragmentDetallePedidoPresenter.eliminarPedidoById(pedidoId, motivo, usuario)
→ PedidoIterator.eliminarPedidoById(...)
→ PedidoRouter → PedidoController.eliminarPedido_SS()
→ Pedido.eliminarPedidoById() → genera ImpresionList
→ return Respuesta(SUCCESS, ..., ImpresionList)
→ Presenter callback:
→ respuesta.ImpresionList.Count > 0?
→ SÍ: PrintUtil.imprimirComandasEthernet(ImpresionList)
→ NO: nada
→ view.showDeletePedido()
```

12.2 Flujo DESPUÉS (con PrinterServices)

```
btnEliminar_Click
→ FragmentDetallePedido.clickbtnEliminar()
→ showModalMotivoAnulacion(usuarioId)
→ FragmentDetallePedidoPresenter.eliminarPedidoById(pedidoId, motivo, usuario)
→ PedidoIterator.eliminarPedidoById(...)
→ PedidoRouter → PedidoController.eliminarPedido_SS()
→ Pedido.eliminarPedidoById() → genera ImpresionList
→ ¿FLAG ON + SERVIDOR?
→ SÍ: PrinterServiceClient.EnviaComandasAsync() (Patrón A Directo)
→ guardarJobsPorComandas() + RegisterJobs() + fix race BD
ImpresionList = vacía
PrintJobResults = [{ jobId: "720", pedidoIds: ["PED-001"] }]
→ NO: retorna ImpresionList con datos (flujo antiguo)
→ return Respuesta(SUCCESS, ..., ImpresionList, PrintJobResults)

→ Presenter callback:
→ respuesta.ImpresionList.Count > 0?
→ SÍ: PrintUtil.imprimirComandasEthernet() (flag OFF)
→ NO + flag ON:
→ verificarPrintJobsAnulacion(respuesta)
→ tieneJobs?
→ SÍ: view.showImprimiendoPrinterService(onSuccess, onFail, cantidadJobs)
→ PSPProgressService(cantidadJobs)
→ ¿YaFinalizado? (race condition: PS procesó antes de crear
PSPProgressService)
→ SÍ + errores: onFail → showFailedPrinterService() (sin modal
progreso)
→ SÍ + sin errores: onSuccess → showSuccessPrinterService() (sin
modal progreso)
→ NO: Mostrar modal progreso, escuchar callbacks:
→ Completado(huboErrores)
→ huboErrores? onFail → showFailedPrinterService()
→ !huboErrores? onSuccess → showSuccessPrinterService()
→ NO: showErrorRegisterView("Servicio de impresión no responde")
```

→ view.showDeletePedido()

PARALELO (si modo cliente):

- PedidoClient recibe respuesta REST con printJobResults
- ClearAndRegisterJobs(printJobResults) ← preserva callbacks tempranos
- Callback :8083 ← PrinterServices notifica resultado
 - PrintJobStatusManager.UpdateStatus()
 - DispararEventosCambio()
 - PSProgressService.OnJobStatusChanged (actualiza barra)
 - PSProgressService.OnAllJobsDone (finaliza)
 - UtilesCabeceraViewModel.OnPrintJobFailed (badge global rojo)

13. Archivos a Modificar (Resumen por migración)

Para cada evento de impresión que migres:

#	Archivo	Qué modificar
1	QuipuNetX/controller/[Controller].cs	Flag guard + enviar a PS + persistir + registrar + inyectar PrintJobResults (Patrón A para comandas, ver §6)
2	(Solo si es multi-documento) QuipuNetX/Services/Print/Strategies/Server[Tipo]Strategy.cs	Crear Strategy si necesita Patrón B (ver §1.4)
3	(Solo si es multi-documento) QuipuNetX/Services/Print/ImpresionStrategyFactory.cs	Registrar Strategy en el switch
4	front/.../[Presenter].cs	Detectar flag + crear verificarPrintJobs[Tipo]() con guards en cada callback onSuccess/onFail (ver §14)
5	(Solo si es un Client nuevo) QuipuNetX/Client/[Client].cs	Parsear printJobResults + ClearAndRegisterJobs()
6	front/.../[IView].cs	Verificar que la interface declare showImprimiendoPrinterService, showFailedPrinterService y showSuccessPrinterService
7	front/.../[View].cs	Implementar los 3 métodos show* + verificar YaFinalizado early-return en showImprimiendoPrinterService (ver §14)
8	QuipuNetX/server/[Server].cs	Propagar ipClienteRest + serializar printJobResults en JSON response
9	QuipuNetX/ws/modules/[Module].cs + WebServer_New.cs	Capturar ipClienteRest del request HTTP y pasarlo al Server

Archivos que NO necesitas tocar (ya funcionan):

Archivo	Razón
PrintJobStatusManager.cs	Ya tiene ClearAndRegisterJobs, UpdateStatus, eventos

PSProgressService.cs	Ya maneja progreso, timeout, catch-up
PrinterServiceUIHelper.cs	Helper centralizado (crear una vez, usar siempre)
UtilesCabeceraViewModel.cs	Ya escucha AnyJobFailed → badge global
UtilesCabecera_New.xaml.cs	Ya tiene lazy-load de Storyboard
PrintJobCallbackServer.cs	Ya recibe callbacks en :8083 (modo cliente)
PrinterServiceClient.cs	Ya envía a PS y retorna PrintJobResults
PedidoprinterjobController.cs	Ya tiene guardarJobsPorTipo() (llamado internamente por la Strategy)

14. Errores Comunes y Cómo Evitarlos

Error	Causa	Solución
PSPProgressService espera 300s sin respuesta	Clear() destruyó estado FAILED del callback	Usar ClearAndRegisterJobs() en modo cliente
Badge global no parpadea en modo cliente	Storyboard null (flag async setea después de Loaded)	Ya fixado: lazy-load en IniciarAnimacionImpresion
Badge no parpadea tras Cleanup/Unloaded	Eventos desuscriptos permanentemente	Ya fixado: re-subscribe en ConfigureImpresiones on Loaded
GetJobStatus retorna null	Clear() borró el job antes que OnPrintJobFailed ejecute	Ya fixado: fallback PrintJobStatusInfo en OnPrintJobFailed
Front imprime localmente aunque flag ON	Falta verificar flag en Presenter	Agregar else if (flag ON) después de ImpresionList.Count > 0
Jobs duplicados en PrintJobStatusManager	RegisterJobs sin Clear previo	ClearAndRegisterJobs limpia batch anterior
Modal de error aparece con jobs viejos	Jobs FAILED de batch anterior en RAM	ClearAndRegisterJobs elimina batch anterior
Modal progreso queda abierto sin transicionar	PS procesó jobs antes de PSPProgressService → Completado nunca se dispara	Verificar psProgress.YaFinalizado antes de crear modal → early return con onSuccess/onFail
NullReferenceException en callback onSuccess/onFail	showDeletePedido(2) dispone la view, y BeginInvoke ejecuta callback después → fragmentDetallePedidoIView es null	Agregar guard if (!_disposed fragmentDetallePedidoIView == null) return; en cada callback
CustomModalImpresionExitosaNew no aparece tras impresión exitosa	onSuccess vacío, o falta showSuccessPrinterService() en la interface/View	Implementar showSuccessPrinterService() en interface + View, y llamarlo desde onSuccess
showSuccessPrinterService no existe en la View	La interface IView no declara el método, o la View no lo implementa	Agregar a IView + implementar en View (patrón: new

		CustomModalImpresionExitosaNew() + auto-close 3s)
--	--	---

15. Testing — Escenarios Mínimos

Modo Servidor

#	Escenario	Flag	Esperado
1a	Anular pedido, PS OK (lento >100ms)	ON	Modal progreso azul → 100% verde → cierra → showSuccessPrinterService (toast verde 3s)
1b	Anular pedido, PS OK (rápido <100ms)	ON	YaFinalizado=true → sin modal progreso → directo a showSuccessPrinterService (toast verde 3s)
2a	Anular pedido, PS falla (lento)	ON	Modal progreso → rojo → showFailedPrinterService → badge global rojo
2b	Anular pedido, PS falla (rápido <100ms)	ON	YaFinalizado=true → sin modal progreso → directo a showFailedPrinterService
3	Anular pedido, PS no responde	ON	showErrorRegisterView("Servicio de impresión no responde") — sin modal progreso
4	Anular pedido, flag OFF	OFF	Impresión local (flujo antiguo, sin cambios)

Modo Cliente

#	Escenario	Flag servidor	Esperado
5	Anular pedido, PS OK	ON	Modal progreso → callback :8083 → 100% verde → cierra
6	Anular pedido, PS falla <5ms	ON	Callback llega antes de PSProgressService → catch-up → showFailedPrinterService → badge global rojo
7	Anular pedido, flag OFF	OFF	Impresión local (flujo antiguo)

Badge global (ambos modos)

#	Escenario	Esperado
8	Cualquier job falla	brdIconoImpresora parpadea rojo con animación pulsante
9	Click en badge	Popup con lista de ImpresionesFallidas
10	Reintento exitoso	Badge desaparece, animación se detiene

16. Prevención de Memory Leaks — Suscripciones y Desuscripciones

16.1 El riesgo

`PrintJobStatusManager` es **singleton**. Todo lo que se suscribe a sus eventos (`JobStatusChanged` , `AnyJobFailed` , `AllJobsDone`) mantiene una referencia fuerte que **impide la recolección de basura (GC)** del suscriptor.

Si un Presenter, ViewModel o View se destruye sin desuscribirse → **memory leak**.

16.2 Inventario de suscriptores y su ciclo de vida

Suscriptor	Tipo	Ciclo de vida	¿Cuándo se desuscribe?	Riesgo
PSPProgressService	Instancia por operación	Se crea al iniciar impresión, muere al finalizar	Desuscribirse() en OnAllJobsDone o Timer_Tick timeout	✅ Bajo — desuscribe en todos los paths
UtilesCabeceraViewModel	Singleton	Toda la vida de la app	Cleanup() en UserControl_Unloaded → re-subscribe en Loaded	⚠️ Medio — corregido con idempotent unsubscribe-before-subscribe
ListaPedidos	Singleton (sInstance re-creado)	Se re-crea al cambiar de mesa	Desuscribe ANTES de reasignar sInstance	⚠️ Medio — la instancia anterior queda si no desuscribe

16.3 Patrón obligatorio: Desuscribir SIEMPRE

Para PSPProgressService (ya implementado):

```
// PSPProgressService.Desuscribirse() – llamado en OnAllJobsDone Y en Timer_Tick (timeout)
private void Desuscribirse()
{
    try
    {
        PrintJobStatusManager.Instance.JobStatusChanged -= OnJobStatusChanged;
        PrintJobStatusManager.Instance.AllJobsDone -= OnAllJobsDone;
        PrintJobStatusManager.Instance.AnyJobFailed -= OnAnyJobFailed;
    }
    catch (Exception) { }
}
```

Paths de desuscripción garantizados:

1. OnAllJobsDone → Desuscribirse() ✅
2. Timer_Tick (timeout) → Desuscribirse() ✅
3. Detener() (dispose manual) → Desuscribirse() ✅

Para UtilesCabeceraViewModel (ya corregido):

```
// Patrón idempotente: desuscribir ANTES de suscribir
// Evita doble suscripción tras Cleanup() → Loaded → Loaded
private void SubscribeToQueueEvents()
{
    // Desuscribir primero (idempotente – no falla si no estaba suscrito)
    PrintJobStatusManager.Instance.AnyJobFailed -= OnPrintJobFailed;
    PrintJobStatusManager.Instance.AllJobsDone -= OnAllJobsDoneViewModel;
```

```
// Suscribirse
PrintJobStatusManager.Instance.AnyJobFailed += OnPrintJobFailed;
PrintJobStatusManager.Instance.AllJobsDone += OnAllJobsDoneViewModel;
}
```

16.4 Patrón para PrinterServiceUIHelper (centralizado)

El `PrinterServiceUIHelper` propuesto en §8 **NO se suscribe a `PrintJobStatusManager`** directamente. Delega a `PSProgressService` que ya tiene su propio ciclo de vida correcto. Esto significa:

```
PrinterServiceUIHelper.MostrarProgresoImpresion()
→ new PSProgressService(cantidadJobs) ← suscribe a 3 eventos
→ psProgress.Completado += handler ← evento local (no singleton)
→ ... jobs terminan ...
→ PSProgressService.Desuscribirse() ← desuscribe 3 eventos ✓
→ PSProgressService queda sin referencias → GC lo recolecta ✓
```

No hay leak porque:

1. `PSProgressService` desuscribe sus 3 handlers en todos los paths de salida
2. El `Completado` event solo tiene 1 handler (el lambda del caller) que muere cuando `PSProgressService` muere
3. El `DispatcherTimer` se detiene explícitamente

16.5 Checklist de memory leaks para nuevas migraciones

- ☐ ¿El código se suscribe a `PrintJobStatusManager` directamente? → **NO hacerlo**. Usar `PSProgressService` o `PrinterServiceUIHelper`.
- ☐ ¿Se crea un `PSProgressService` sin llegar a `Desuscribirse()`? → Verificar que **todos los paths** (éxito, error, timeout, dispose) llamen `Desuscribirse()` o `Detener()`.
- ☐ ¿Se suscribe a `Completado` con lambda? → El lambda captura variables del closure. Si el closure referencia a la View, verificar que la View no se destruya antes que `Completado` se dispare.
- ☐ ¿La View implementa `IDisposable`? → En `Dispose()`, llamar `psProgress?.Detener()` si hay referencia.
- ☐ ¿El Presenter tiene `_disposed` flag? → Verificar en callbacks antes de acceder a la View.

16.6 Diagrama de ciclo de vida de suscripciones

PERMANENTES (viven toda la app):

```
UtilesCabeceraViewModel —subscribe→ PrintJobStatusManager.AnyJobFailed
UtilesCabeceraViewModel —subscribe→ PrintJobStatusManager.AllJobsDone
ListaPedidos (sInstance) —subscribe→ PrintJobStatusManager.AnyJobFailed
ListaPedidos (sInstance) —subscribe→ PrintJobStatusManager.JobStatusChanged
|
├ Se desuscriben en Cleanup()/Unloaded
└ Se RE-suscriben en Loaded (idempotente)
```

EFÍMERAS (viven por operación de impresión):

```
PSProgressService —subscribe→ PrintJobStatusManager.JobStatusChanged
PSProgressService —subscribe→ PrintJobStatusManager.AllJobsDone
PSProgressService —subscribe→ PrintJobStatusManager.AnyJobFailed
|
├ Se desuscriben en OnAllJobsDone (éxito/error)
├ Se desuscriben en Timer_Tick (timeout)
└ Se desuscriben en Detener() (dispose manual)
```